

Exposé – Abschlussarbeit IKT-Master

Konzeption und Implementierung eines LLM-Tutors für digitale Mathematikaufgaben

Sidney Göhler

24. Juli 2026

Zusammenfassung

Die Arbeit untersucht die Konzeption und Implementierung eines LLM-basierten Tutors für digitale Mathematikaufgaben. Als technische Ausgangsbasis dienen Moodle-STACK-Aufgaben aus dem Themenbereich Differentialrechnung. STACK und das zugrunde liegende Computer-Algebra-System Maxima übernehmen die symbolische Bewertung mathematischer Eingaben, während ein selbst gehostetes Large Language Model adaptive didaktische Hinweise generiert. Die Tutorsteuerung erfolgt über einen expliziten Prompt-Workflow mit gestuften Hilfen und Regeln zur Vermeidung unmittelbaren Lösungs verrats. Zusätzlich wird untersucht, in welcher Form Maxima beziehungsweise eine Headless-STACK-Instanz als mathematisches Prüfwerkzeug oder als aufrufbares Werkzeug eines LLM-Agenten eingebunden werden kann. Die technische Orchestrierung soll prototypisch über eine Workflow-Plattform wie n8n erfolgen. Der Prototyp wird anhand eines Satzes von Differentialrechnungsaufgaben mit typischen Fehlern evaluiert. Optional ist eine explorative Erprobung mit Studierenden mittels Think-Aloud-Methode vorgesehen.

Inhaltsverzeichnis

Inhaltsverzeichnis	2
1 Ausgangslage und Problemstellung	4
2 Forschungsfrage	6
3 Zielsetzung	8
4 Untersuchungsgegenstand und Eingrenzung	10
5 Methodisches Vorgehen	12
5.1 Literatur- und Marktanalyse	12
5.2 Analyse der Anforderungen	13
5.3 Auswahl und Modellierung des Aufgabensatzes	14
5.4 Entwicklung des technischen Prototyps	15
5.5 Anbindung von Maxima und Headless STACK	16
5.6 Workflow-Orchestrierung mit n8n	16
5.7 Kontext- und Wissensbasis	17
5.8 Prompt-, Workflow- und Tutor-Design	18
5.9 Auswahl eines selbst gehosteten LLMs	19
5.10 Evaluation	20
6 Erwartete Ergebnisse und wissenschaftlicher Beitrag	23
7 Quellenlage / aktueller Stand der Technik	24
7.1 Mathematisches E-Assessment und STACK	24
7.2 LLM-basierte Mathematiktutoren und Hint-Generierung	24
7.3 Retrieval-Augmented Generation und Kontextsteuerung	25
7.4 Symbolische Verifikation und neuro-symbolische Ansätze	25
7.5 Kommerzielle und institutionelle Systeme	26
7.6 Forschungslücke	26
8 Zeitplan	27
9 Vorläufige Gliederung	29

◀ 1 Ausgangslage und Problemstellung

Digitale Übungs- und Prüfungssysteme sind in mathematischen Grundlagenveranstaltungen an Hochschulen fest etabliert. Insbesondere das Open-Source-E-Assessment-System **STACK**, das als Fragetyp in Moodle eingesetzt werden kann, ermöglicht die symbolische Bewertung mathematischer Eingaben auf Basis des Computer-Algebra-Systems Maxima [San13; STA26]. Dadurch können nicht nur numerische Endergebnisse, sondern auch algebraisch äquivalente Ausdrücke erkannt und bewertet werden. Über sogenannte **Potential Response Trees** (PRTs) können Lehrende typische Fehlermuster modellieren, durch symbolische Tests identifizieren und mit spezifischem Feedback versehen [STA26].

Trotz dieser Stärken bleibt das Feedback in STACK-Aufgaben in der Regel vorab definiert. Es muss durch Lehrende für einzelne Antwort- und Fehlerfälle formuliert werden und kann nur begrenzt flexibel auf individuelle Lösungswege, Ausdrucksweisen oder Verständnisschwierigkeiten reagieren. Formative Rückmeldungen gelten insbesondere dann als lernförderlich, wenn sie den nächsten Handlungsschritt unterstützen, verständlich formuliert sind und die Selbstregulation der Lernenden fördern [HT07; Shu08; NM06]. Aktuelle Large Language Models (LLMs) eröffnen demgegenüber neue Möglichkeiten für adaptive, sprachlich differenzierte und dialogorientierte Lernunterstützung. LLMs können Erklärungen generieren, aktivierende Rückfragen formulieren und Hinweise an den jeweiligen Bearbeitungsstand anpassen.

Der Einsatz von LLMs in mathematischen Lernumgebungen ist jedoch mit spezifischen Risiken verbunden. Dazu zählen insbesondere fachlich fehlerhafte Aussagen, Halluzinationen, unzuverlässige mathematische Bewertungen und die unmittelbare Ausgabe vollständiger Lösungen [Kas+23; Ji+23; NMC23]. Ein LLM sollte daher nicht als alleinige mathematische Bewertungsinstanz eingesetzt werden. Stattdessen erscheint eine neuro-symbolische Aufgabenteilung sinnvoll: Ein deterministisches mathematisches System prüft die formale Korrektheit, während das LLM die didaktische und sprachliche Aufbereitung übernimmt. Aktuelle Arbeiten zeigen, dass die Kombination von LLMs mit symbolischen Berechnungs- und Verifikationskomponenten mathematische Fehler

◀1 Ausgangslage und Problemstellung

und Halluzinationen reduzieren kann [IDS23; Meu+17; Wan26].

Die geplante Arbeit setzt an dieser Schnittstelle an. STACK und Maxima sollen die mathematische Bewertung und gegebenenfalls Fehlerdiagnose übernehmen. Ein externes, selbst gehostetes LLM soll auf Basis der Aufgabeninformationen, der Studierendenantwort und verfügbarer Prüfergebnisse adaptive didaktische Hinweise generieren. Untersucht wird dabei auch, ob Maxima oder eine Headless-STACK-Instanz als aufrufbares Werkzeug in den Tutor-Workflow integriert werden kann. Eine solche Anbindung könnte über eine definierte Schnittstelle oder agentenähnliche Werkzeugnutzung erfolgen und zur Prüfung mathematischer Ausdrücke, Zwischenschritte oder generierter Tutorhinweise eingesetzt werden.

Im Unterschied zu Ansätzen, bei denen ein symbolischer Solver ausschließlich vom LLM erzeugte Lösungen nachträglich prüft, soll die symbolische Komponente hier insbesondere die Studierendenantwort bewerten und eine kontrollierte Grundlage für die tutorielle Hinweisgenerierung liefern.

Die Orchestrierung der einzelnen Komponenten kann prototypisch über eine Workflow-Plattform wie *n8n* realisiert werden [n8n26]. Ein möglicher Workflow umfasst die Entgegennahme von STACK-Daten, die Auswahl aufgabenspezifischer Kontextinformationen, den optionalen Aufruf von Maxima beziehungsweise Headless STACK, die Erzeugung eines strukturierten Prompts, den Aufruf des selbst gehosteten LLMs und die Rückgabe eines Tutorhinweises.

Als fachliche Anwendungsdomäne wird die **Differentialrechnung** gewählt. Damit wird die Untersuchung nicht ausschließlich auf die Kettenregel oder verkettete Exponentialfunktionen beschränkt, sondern auf einen überschaubaren Satz typischer Ableitungsaufgaben erweitert. Dieser kann beispielsweise Potenz-, Produkt-, Quotienten- und Kettenregel sowie Kombinationen dieser Regeln umfassen.

Die korrekte Bearbeitung erfordert unter anderem die Anwendung der Kettenregel, die Ableitung des inneren Exponenten und die Berücksichtigung des konstanten Faktors. Typische Fehler können entweder durch einen entsprechend modellierten PRT erkannt oder alternativ durch einen mathematischen Prüfworkflow mit Maxima beziehungsweise Headless STACK analysiert werden. Ein Vergleich beider Ansätze ist als optionale Untersuchung vorgesehen.

◀ 2 Forschungsfrage

Die zentrale Forschungsfrage der Arbeit lautet:

Wie kann ein selbst gehosteter LLM-Tutor für digitale Mathematikaufgaben so konzipiert und implementiert werden, dass er Studierende auf Grundlage symbolischer mathematischer Prüfungen durch adaptive Hinweise unterstützt, ohne die vollständige Lösung unmittelbar vorzugeben?

Aus dieser übergeordneten Forschungsfrage ergeben sich folgende Teilfragen:

1. **Informationsübergabe und Systemarchitektur**
Welche Informationen müssen aus einer digitalen Mathematikaufgabe an den LLM-Tutor übergeben werden, um aufgaben- und fehlerbezogene Hilfestellungen generieren zu können?
2. **Symbolische Prüfung und Fehlerdiagnose**
Wie können STACK, Potential Response Trees, Maxima oder eine Headless-STACK-Instanz zur Prüfung von Studierendenantworten und gegebenenfalls zur Absicherung generierter Tutorhinweise eingesetzt werden?
3. **Vergleich von Diagnoseansätzen**
Welche Unterschiede ergeben sich zwischen einer vorab durch Lehrende modellierten Fehlerdiagnose mittels PRT und einer dynamischen mathematischen Analyse durch Maxima beziehungsweise Headless STACK?
4. **Prompt- und Workflow-Design**
Wie müssen Prompt, Tutor-Policy und technische Prozesssteuerung gestaltet sein, damit das LLM fachlich korrekte, verständliche und didaktisch angemessene Hinweise erzeugt?
5. **Vermeidung von Lösungsverrat**
Wie können gestufte Hilfen und ein im Prompt hinterlegter Tutor-Workflow das Risiko reduzieren, dass das LLM unmittelbar die vollständige Lösung ausgibt?

◀2 Forschungsfrage

6. Modellwahl

Welches selbst gehostete LLM eignet sich hinsichtlich mathematischer Zuverlässigkeit, deutschsprachiger Ausdrucksqualität, Befolgung der Tutor-Policy und Antwortzeit für den betrachteten Anwendungsfall?

7. Technische und didaktische Qualität

Wie zuverlässig funktioniert der Prototyp bei einem Satz typischer Differentialrechnungsaufgaben und wie sind die generierten Hinweise hinsichtlich fachlicher Korrektheit, Fehlerbezug, Verständlichkeit, Hilfreichkeit und Lösungsverrat zu bewerten?

◀ 3 Zielsetzung

Ziel der Arbeit ist die Konzeption, prototypische Implementierung und Evaluation eines LLM-Tutors für digitale Mathematikaufgaben aus dem Themenbereich Differentialrechnung.

Der Prototyp soll eine Informationskette zwischen einer digitalen Aufgabe, einem symbolischen mathematischen Prüfsystem und einem selbst gehosteten LLM realisieren. STACK und Maxima übernehmen dabei die formale mathematische Bewertung. Das LLM übernimmt die sprachliche und didaktische Formulierung von Hinweisen. Hierzu sollen Aufgabeninformationen, Studierendenantworten, Prüfergebnisse und gegebenenfalls PRT-Diagnosecodes in einem kontrollierten Workflow verarbeitet werden.

Konkret verfolgt die Arbeit folgende Ziele:

- Analyse wissenschaftlicher Arbeiten und bestehender kommerzieller Systeme zu digitalen Mathematikutoren, LLM-basiertem Feedback, RAG, symbolischer Verifikation und intelligenten Tutorsystemen
- Entwicklung eines technischen Interfaces zur Übergabe relevanter Informationen zwischen STACK, Tutor-Workflow, symbolischem Prüfsystem und LLM
- prototypische Anbindung von Maxima beziehungsweise Headless STACK zur mathematischen Prüfung von Antworten oder generierten Ausdrücken
- Untersuchung einer optionalen Werkzeug- beziehungsweise Agentenschnittstelle, über die das LLM definierte mathematische Prüfoperationen anfordern kann
- Orchestrierung des Tutor-Workflows, beispielsweise mit n8n
- Auswahl und Erprobung mindestens eines selbst gehosteten LLMs
- Entwicklung eines Prompt-Workflows mit gestuften Hilfen und expliziten Regeln zur Vermeidung von Lösungserrat
- Vergleich unterschiedlicher Kontext- und Diagnosebedingungen

◀3 Zielsetzung

- technische Evaluation anhand eines kuratierten Aufgabensatzes mit typischen Fehlerantworten
- optional Einbeziehung anonymisierter realer Studierendenantworten, sofern diese rechtlich und organisatorisch verfügbar sind
- optional explorative Erprobung mit Studierenden mittels Think-Aloud-Methode
- Ableitung von Gestaltungsempfehlungen für LLM-basierte Mathematik-tutoren

Die Arbeit ist ausdrücklich als prototypische Untersuchung angelegt. Eine vollständige produktive Integration in Moodle, eine umfassende Rechteverwaltung, eine großangelegte empirische Wirksamkeitsstudie und die Abdeckung beliebiger mathematischer Themen sind nicht Gegenstand der Arbeit.

◀ 4 Untersuchungsgegenstand und Eingrenzung

Untersuchungsgegenstand ist ein selbst gehosteter LLM-Tutor für digitale Mathematikaufgaben, der mit einem symbolischen mathematischen Prüfsystem verbunden wird.

Die Eingrenzung erfolgt auf:

- mathematisches Thema: Differentialrechnung
- Aufgabentypen: ausgewählte Aufgaben zu grundlegenden Ableitungsregeln
- mathematische Prüfung: STACK/Maxima sowie optional Headless STACK
- Modellbetrieb: selbst gehostetes LLM
- technische Orchestrierung: prototypischer Workflow, beispielsweise über n8n
- didaktischer Fokus: adaptive und gestufte Hinweise ohne unmittelbare Lösungsvorgabe
- Evaluationsfokus: technische Funktionsfähigkeit und Qualität der Tutorhinweise

Nicht betrachtet werden:

- ein vollständiges produktives Moodle-Plugin
- eine allgemeine Tutoring-Plattform für beliebige mathematische Fachgebiete
- die alleinige mathematische Bewertung durch das LLM
- umfangreiche Learning-Analytics-Funktionen
- eine großangelegte quantitative Wirksamkeitsstudie

◀4 Untersuchungsgegenstand und Eingrenzung

- die Implementierung einer umfangreichen embedding-basierten Vektordatenbank

Auf eine umfangreiche Vektordatenbank wird im Prototyp verzichtet. Da der Untersuchungsbereich auf einen kuratierten Satz von Aufgaben aus der Differentialrechnung beschränkt ist und mit Aufgaben-ID, Aufgabentyp, Diagnosecode und Hilfestufe bereits strukturierte Auswahlmerkmale vorliegen, kann der relevante Kontext deterministisch aus einer datei- oder datenbankbasierten Wissensbasis geladen werden. Eine semantische Vektorsuche wird als mögliche Erweiterung für größere und heterogene Aufgabenbestände diskutiert.

◀ 5 Methodisches Vorgehen

Die Arbeit folgt einem entwicklungsorientierten Forschungsansatz. Im Mittelpunkt stehen die Konzeption, Implementierung und Evaluation eines technischen Artefakts. Das Vorgehen umfasst eine Literatur- und Marktanalyse, eine Anforderungsanalyse, die prototypische Softwareentwicklung sowie eine technische und didaktisch-qualitative Evaluation.

5.1 Literatur- und Marktanalyse

Zu Beginn wird der wissenschaftliche und technische Stand analysiert. Die Literaturrecherche umfasst Arbeiten zu mathematischem E-Assessment, intelligenten Tutorsystemen, LLM-basierter Lernunterstützung, Retrieval-Augmented Generation, symbolischer Verifikation und formativem Feedback. STACK und computeralgebrabasierte Bewertung werden insbesondere anhand der grundlegenden Darstellung von Sangwin [San13] sowie der aktuellen STACK-Dokumentation betrachtet [STA26].

Die Literaturrecherche umfasst insbesondere:

- STACK und computeralgebrabasiertes E-Assessment
- intelligente Tutorsysteme und automatische Hint-Generierung
- LLM-basierte Mathematiktutoren
- Retrieval-Augmented Generation und deterministische Kontextanreicherung
- neuro-symbolische Systeme und symbolische Solver-Rückkopplung
- Prompt-Design und gestufte Tutorhinweise
- formative Rückmeldung und Vermeidung von Lösungsverrat
- Think-Aloud als qualitative Evaluationsmethode

Für die didaktische Gestaltung der Hinweise werden insbesondere Arbeiten zu formativem Feedback und intelligenten Tutorsystemen berücksichtigt.

◀5 Methodisches Vorgehen

VanLehn [Van06] beschreibt gestufte Hint-Sequenzen als wesentliches Element tutorieller Systeme. Weitere Arbeiten untersuchen die automatische Generierung, Nutzung und Gestaltung von Hinweisen [Sta+10; RH10; Koc+22; Man+20].

Für die Kontextanreicherung sind Arbeiten zu Retrieval-Augmented Generation relevant. Das grundlegende RAG-Verfahren wurde von Lewis u. a. [Lew+20] vorgestellt. Aktuelle Übersichtsarbeiten ordnen unterschiedliche Retrieval-, Generierungs- und Evaluationsstrategien ein [Gao+23; Mia+23]. Für mathematische Lernunterstützung ist insbesondere die Untersuchung von Levonian u. a. [Lev+23] relevant. Sie zeigt, dass RAG die curriculare Verankerung mathematischer Tutorantworten verbessern kann, eine besonders starke Bindung an abgerufene Materialien jedoch nicht zwangsläufig zu einer höheren menschlichen Präferenz führt.

Aktuelle neuro-symbolische Arbeiten kombinieren LLMs mit externen Berechnungs- oder Verifikationswerkzeugen. Dazu zählen der Einsatz eines Code-Scratchpads für mathematisches Tutoring [UGC23], symbolische Berechnungen mit SymPy [Meu+17] und die Verbindung eines mathematischen RAG-Systems mit symbolischer Solver-Rückkopplung [Wan26]. Diese Arbeiten bilden eine wichtige Grundlage für die geplante Anbindung von Maxima beziehungsweise Headless STACK.

Ergänzend werden bestehende kommerzielle und institutionelle Angebote betrachtet. Dazu zählen allgemeine LLM-Systeme, mathematikspezifische Solver, adaptive Lernplattformen sowie hochschulbezogene KI-Tutoren. Als mögliche Referenzsysteme werden unter anderem Khanmigo [Kha26], WolframAlpha [Wol26], Möbius [Map26] und das Angebot *KI & ME* der Hochschule Magdeburg-Stendal [Hoc26] betrachtet. Ziel ist eine Einordnung der geplanten Lösung und keine umfassende Marktstudie.

5.2 Analyse der Anforderungen

Auf Grundlage der Literatur- und Marktanalyse werden die funktionalen, didaktischen und technischen Anforderungen an den Prototyp festgelegt. Betrachtet werden insbesondere folgende Fragen:

- Welche Informationen können aus STACK oder einer vergleichbaren digitalen Aufgabe bereitgestellt werden?
- Welche Informationen benötigt das LLM, um einen hilfreichen Hinweis zu erzeugen?
- Welche mathematischen Prüfoperationen müssen durch Maxima oder Headless STACK bereitgestellt werden?
- Welche Informationen dürfen auf den jeweiligen Hilfestufen verwendet werden?

◀5 Methodisches Vorgehen

- Wie soll der Tutor reagieren, wenn keine verlässliche Diagnose vorliegt?
- Wie werden fehlerhafte, manipulierte oder unzureichende Eingaben behandelt?
- Welche Anforderungen bestehen an Datenschutz, Nachvollziehbarkeit und Antwortzeit?

Das Ergebnis ist ein Anforderungskatalog für Systemarchitektur, Tutor-Workflow und Evaluation.

5.3 Auswahl und Modellierung des Aufgabensatzes

Für die Evaluation wird ein begrenzter Aufgabensatz aus der Differentialrechnung zusammengestellt. Dieser soll unterschiedliche Ableitungsregeln und typische Fehlerklassen abdecken, beispielsweise:

- Potenzregel
- Produktregel
- Quotientenregel
- Kettenregel
- Ableitung von Exponential- und trigonometrischen Funktionen
- Kombination mehrerer Ableitungsregeln

Zu jeder Aufgabe werden typische fehlerhafte Antworten definiert. Diese können aus didaktischer Literatur, der Erfahrung von Lehrenden, vorhandenen PRT-Strukturen und, sofern verfügbar, anonymisierten realen Studierendenantworten abgeleitet werden.

Eine exemplarische Aufgabe lautet:

$$f(x) = -5e^{x^2-2e^x}.$$

Mögliche Fehler sind:

- fehlende innere Ableitung
- fehlerhafte Ableitung von e^x
- fehlerhafte Ableitung von x^2
- vergessener konstanter Faktor

◀5 Methodisches Vorgehen

- Vorzeichenfehler
- Ableitung ausschließlich des inneren Exponenten

In einer STACK-Aufgabe werden solche Fehlermuster durch die Autorinnen und Autoren im PRT modelliert. Der PRT identifiziert die Fehler anschließend durch symbolische Tests. Optional wird untersucht, inwiefern eine dynamische Analyse mit Maxima oder Headless STACK vergleichbare beziehungsweise ergänzende Informationen erzeugen kann.

5.4 Entwicklung des technischen Prototyps

Der Prototyp soll folgende Informations- und Verarbeitungskette realisieren:

digitale Mathematikaufgabe → Tutor-Workflow
→ STACK/Maxima oder Headless STACK
→ selbst gehostetes LLM
→ Tutor-Interface.

Für die prototypische Moodle-STACK-Anbindung kann ein Feedback-Link verwendet werden, der eine Aufgaben-ID, die Studierendenantwort und gegebenenfalls einen PRT-Diagnosecode an das Tutor-Interface übergibt. Eine direkte Rückintegration des Hinweises in die ursprüngliche STACK-Oberfläche ist optional und nicht zwingende Voraussetzung des Minimalprototyps.

Der Prototyp soll mindestens folgende Funktionen bereitstellen:

- Entgegennahme einer Aufgaben-ID und Studierendenantwort
- Entgegennahme oder Ermittlung eines Diagnosecodes
- Validierung der übertragenen Daten
- Laden aufgabenspezifischer Kontextinformationen
- Aufruf von Maxima beziehungsweise Headless STACK zur mathematischen Prüfung
- Erzeugung eines strukturierten Tutor-Prompts
- Aufruf eines selbst gehosteten LLMs
- Anzeige oder Rückgabe des generierten Hinweises
- Protokollierung der für die Evaluation erforderlichen technischen Daten

◀5 Methodisches Vorgehen

5.5 Anbindung von Maxima und Headless STACK

Maxima beziehungsweise Headless STACK soll als deterministische mathematische Komponente in den Tutor-Workflow eingebunden werden. Untersucht werden können folgende Varianten:

1. **PRT-basierte Diagnose**

STACK bewertet die Antwort und übergibt einen durch den PRT bestimmten Diagnosecode an den Tutor.

2. **Direkter Maxima-Aufruf**

Der Workflow übermittelt mathematische Ausdrücke an eine Maxima-Schnittstelle und erhält beispielsweise Ergebnisse zu Äquivalenz, Differenz, Vereinfachung oder Ableitung.

3. **Headless-STACK-Prüfung**

Eine STACK-Prüfumgebung wird ohne Moodle-Oberfläche aufgerufen, um vorhandene STACK-Tests und PRT-Logik wiederzuverwenden.

4. **Werkzeugnutzung durch einen Agenten**

Das LLM kann innerhalb eines streng begrenzten Workflows definierte mathematische Prüfwerkzeuge anfordern. Die Werkzeugergebnisse werden anschließend für die Formulierung des Hinweises verwendet.

Die agentische Variante wird nur umgesetzt, sofern sie innerhalb des zeitlichen Rahmens einen erkennbaren Mehrwert gegenüber einem deterministischen Workflow bietet. Das LLM erhält keine uneingeschränkte Ausführungs- oder Bewertungsbefugnis.

5.6 Workflow-Orchestrierung mit n8n

Die technische Prozesssteuerung kann mit *n8n* prototypisch umgesetzt werden. Ein möglicher Workflow umfasst:

1. Entgegennahme der Anfrage aus STACK
2. Prüfung und Normalisierung der Eingabeparameter
3. Laden der Aufgaben- und Tutor-Metadaten
4. Aufruf von Maxima oder Headless STACK
5. Auswahl der Hilfestufe und des zugehörigen Prompt-Workflows
6. Aufruf des selbst gehosteten LLMs
7. optionale mathematische oder regelbasierte Prüfung der LLM-Ausgabe

◀5 Methodisches Vorgehen

8. Rückgabe des Tutorhinweises
9. Speicherung technischer Messwerte für die Evaluation

Dabei ist zu prüfen, ob n8n als vollständige Orchestrierungsschicht verwendet wird oder ob einzelne Funktionen in einer ergänzenden Python-Anwendung implementiert werden. Die Entscheidung erfolgt anhand von Nachvollziehbarkeit, Wartbarkeit, Funktionsumfang und Implementierungsaufwand.

5.7 Kontext- und Wissensbasis

Die Wissensbasis wird für den begrenzten Aufgabensatz kuratiert und kann folgende Bestandteile enthalten:

- Aufgabentext und Aufgaben-ID
- mathematisches Thema und Lernziel
- relevante Ableitungsregeln
- Musterlösung und geprüfte Zwischenschritte
- typische Fehler und PRT-Diagnosecodes
- geeignete Tutorstrategien
- Regeln für die einzelnen Hilfestufen
- zulässige und unzulässige Inhalte eines Hinweises

Die Kontextauswahl erfolgt voraussichtlich deterministisch anhand von Aufgaben-ID, Diagnosecode und Hilfestufe. Dieser Ansatz reduziert technische Komplexität und vermeidet zusätzliche Retrieval-Fehler bei einer kleinen, strukturierten Wissensbasis. Der Ansatz kann als retrieval-gestützte Kontextanreicherung beschrieben werden; eine klassische embedding-basierte RAG-Architektur mit Vektordatenbank ist nicht zwingender Bestandteil des Prototyps.

Das Vorgehen greift die Grundidee von Retrieval-Augmented Generation auf, externes und kuratiertes Wissen zur Generierungszeit bereitzustellen [Lew+20]. Aufgrund des begrenzten und strukturierten Aufgabenbestands wird jedoch auf eine embedding-basierte Vektorsuche verzichtet. Die Kontextauswahl erfolgt stattdessen deterministisch anhand bereits verfügbarer Metadaten. Diese Entscheidung soll die Reproduzierbarkeit erhöhen und zusätzliche Retrieval-Fehler vermeiden.

◀5 Methodisches Vorgehen

Die Wissensbasis kann sowohl deklaratives mathematisches Wissen als auch Anwendungsbeispiele enthalten. Aktuelle Arbeiten zu anwendungsorientiertem RAG zeigen, dass die gemeinsame Bereitstellung von Regelwissen und Beispielen seiner Anwendung mathematische Reasoning-Aufgaben verbessern kann [Wan+25]. Für den Tutor-Prototyp bedeutet dies, dass neben Ableitungsregeln auch typische Fehlerantworten, PRT-Diagnosen und geeignete Tutorstrategien hinterlegt werden können.

5.8 Prompt-, Workflow- und Tutor-Design

Ein Schwerpunkt der Arbeit liegt auf dem Entwurf des Tutor-Workflows. Die Hilfestufen werden nicht ausschließlich als Metadaten übergeben, sondern explizit als Handlungslogik im Prompt beziehungsweise im n8n-Workflow hinterlegt.

Die gestufte Tutorsteuerung orientiert sich an etablierten Hint-Sequenzen intelligenter Tutorsysteme. Hinweise sollen schrittweise konkreter werden, wobei zunächst die Aufmerksamkeit auf den relevanten Aufgabenteil gelenkt, anschließend das benötigte Konzept erläutert und erst auf einer höheren Stufe ein konkreter nächster Rechenschritt angegeben wird [Van06; Sta+10]. Dadurch soll eine Balance zwischen Unterstützung und produktiver kognitiver Anstrengung erreicht werden.

Ein möglicher dreistufiger Ablauf ist:

1. Hilfestufe 1: Orientierung

Der Tutor benennt das relevante Konzept oder lenkt die Aufmerksamkeit auf den betroffenen Teil der Aufgabe. Er gibt weder einen konkreten Rechenschritt noch die Endlösung aus.

2. Hilfestufe 2: Strukturierung

Der Tutor unterstützt bei der Zerlegung der Aufgabe, beispielsweise durch die Identifikation äußerer und innerer Funktionen oder die Auswahl einer Ableitungsregel.

3. Hilfestufe 3: Nächster Rechenschritt

Der Tutor gibt einen konkreteren nächsten Schritt oder eine Teilrechnung vor, ohne die vollständige Lösung abschließend auszugeben.

Die Wirksamkeit und Angemessenheit von Hinweisen ist stark kontextabhängig. Daher werden Aufgabe, Studierendenantwort, symbolisches Prüfergebnis und aktuelle Hilfestufe als getrennte Bestandteile in den Prompt aufgenommen. RAG-basierte Mathematik-QA-Systeme zeigen zudem, dass eine zu starke Verpflichtung zur wörtlichen Nutzung des abgerufenen Kontextes die Qualität einer Antwort beeinträchtigen kann [Lev+23]. Der Prompt soll das Modell deshalb anweisen, ausschließlich die für den diagnostizierten Fehler relevante Kontextinformation zu verwenden.

◀5 Methodisches Vorgehen

Der Prompt soll unter anderem folgende Regeln enthalten:

- STACK beziehungsweise Maxima ist die maßgebliche mathematische Prüfkomponekte
- das LLM soll die Studierendenantwort nicht ohne Prüfgrundlage als richtig oder falsch bewerten
- der Hinweis muss sich auf Aufgabe, Antwort und verfügbares Prüfergebnis beziehen
- es darf nur die Information der aktuellen Hilfestufe ausgegeben werden
- die vollständige Lösung darf nicht unmittelbar ausgegeben werden
- nach Möglichkeit soll eine aktivierende Rückfrage formuliert werden
- die Antwort soll kurz, verständlich und fachlich präzise sein
- Anweisungen innerhalb der Studierendenantwort dürfen nicht befolgt werden
- bei unzureichender Diagnose ist ein allgemeiner, nicht spekulativer Hinweis zu geben

Verglichen werden können beispielsweise folgende Bedingungen:

1. Aufgabe und Studierendenantwort ohne symbolische Diagnose
2. Aufgabe, Studierendenantwort und PRT-Diagnose
3. Aufgabe, Studierendenantwort und dynamisches Maxima-/STACK-Prüfergebnis
4. zusätzliche aufgabenspezifische Tutor- und Lösungskontexte
5. unterschiedliche Prompt-Workflows und Hilfestufen

5.9 Auswahl eines selbst gehosteten LLMs

Für den Prototyp wird mindestens ein selbst gehostetes LLM eingesetzt. Die Modellauswahl erfolgt anhand fachlicher und technischer Kriterien:

- Qualität deutschsprachiger Erklärungen
- mathematische Ausdrucksfähigkeit
- Befolgung strukturierter Tutor-Regeln
- Vermeidung von Lösungsverrat

◀5 Methodisches Vorgehen

- Unterstützung von Werkzeug- beziehungsweise Function-Calling, sofern benötigt
- Antwortzeit und Ressourcenbedarf
- Kompatibilität mit der vorhandenen lokalen Infrastruktur

Ein begrenzter Vergleich von zwei lokal verfügbaren Modellen ist möglich. Ein umfassender Modellbenchmark ist nicht Ziel der Arbeit. Der Vergleich mit kommerziellen Systemen erfolgt vorrangig im Rahmen der Markt- und Literaturanalyse; ein experimenteller Vergleich wird nur durchgeführt, wenn Zeit, Zugriffsbedingungen und Datenschutzanforderungen dies zulassen.

5.10 Evaluation

Die Evaluation besteht aus einer technischen sowie einer fachlich-didaktischen Komponente.

Technische Evaluation

Für die technische Evaluation wird ein Satz von Differentialrechnungsaufgaben mit typischen korrekten und fehlerhaften Antworten zusammengestellt. Soweit verfügbar und datenschutzrechtlich zulässig, können anonymisierte reale Studierendenantworten einbezogen werden. Andernfalls werden plausible Fehlerantworten auf Grundlage von PRT-Strukturen, Lehrendenerfahrung und Literatur konstruiert.

Geprüft werden insbesondere:

- korrekte Übergabe der Informationen aus STACK
- korrekte Kodierung mathematischer Eingaben
- korrekte Zuordnung von Aufgaben-ID und Kontext
- korrekte Erkennung beziehungsweise Übergabe des PRT-Diagnosecodes
- Funktionsfähigkeit der Maxima- oder Headless-STACK-Anbindung
- Übereinstimmung symbolischer Prüfergebnisse mit erwarteten Ergebnissen
- korrekte Ausführung des n8n-Workflows
- korrekte Auswahl der Hilfestufe
- erfolgreiche Kommunikation mit dem selbst gehosteten LLM
- Stabilität bei syntaktisch fehlerhaften und unerwarteten Eingaben

◀5 Methodisches Vorgehen

- Antwortzeiten der einzelnen Systemkomponenten und des Gesamtworkflows

Optional wird verglichen, wie zuverlässig typische Fehler durch vorab modellierte PRT-Zweige und durch eine dynamische Maxima-/Headless-STACK-Analyse erkannt werden.

Fachlich-didaktische Evaluation

Die generierten Tutorhinweise werden anhand eines Kriterienrasters bewertet. Mögliche Kriterien sind:

- fachliche Korrektheit
- Übereinstimmung mit dem symbolischen Prüfergebnis
- Passung zur Studierendenantwort
- Passung zum PRT-Diagnosecode beziehungsweise Fehlerfall
- Verständlichkeit
- Hilfreichkeit für den nächsten Lösungsschritt
- Angemessenheit der Hilfestufe
- Vermeidung von Lösungsverrat
- Aktivierung der Lernenden durch Rückfragen
- sprachliche Präzision

Die Bewertung kann durch Lehrende oder Tutorinnen und Tutoren erfolgen. Für zentrale Kriterien können sowohl Likert-Skalen als auch binäre Merkmale verwendet werden, beispielsweise:

- enthält eine mathematisch falsche Aussage: ja/nein
- enthält die vollständige Lösung: ja/nein
- bezieht sich auf den diagnostizierten Fehler: ja/nein
- entspricht der vorgesehenen Hilfestufe: ja/nein

◀5 Methodisches Vorgehen

Optionale Think-Aloud-Erprobung

Optional kann eine kleine explorative Erprobung mit Studierenden durchgeführt werden. Hierfür eignet sich die **Think-Aloud-Methode**, bei der Versuchspersonen ihre Gedanken während der Aufgabenbearbeitung verbalisieren [ES93; SBS94]. Dabei bearbeiten Studierende ausgewählte Aufgaben, erhalten generierte Hinweise und verbalisieren ihre Interpretation des Hinweises sowie die daraus folgenden Lösungsschritte.

Mögliche Leitfragen sind:

- War der Hinweis verständlich?
- Hat der Hinweis geholfen, den eigenen Fehler zu erkennen?
- Unterstützte der Hinweis einen eigenständigen nächsten Schritt?
- War der Hinweis zu allgemein, angemessen oder zu direkt?
- Wurde die Lösung vorweggenommen?
- Würde ein solcher Tutor beim selbstständigen Üben als hilfreich empfunden?

Die qualitative Untersuchung dient nicht dem Nachweis allgemeiner Lernwirksamkeit, sondern der explorativen Einschätzung von Verständlichkeit, Nützlichkeit und didaktischer Angemessenheit.

◀ 6 Erwartete Ergebnisse und wissenschaftlicher Beitrag

Als Ergebnis der Arbeit wird ein funktionsfähiger Prototyp erwartet, der eine digitale Mathematikaufgabe mit einem symbolischen Prüfsystem und einem selbst gehosteten LLM-Tutor verbindet. Zusätzlich sollen ein definierter Tutor-Workflow, Prompt-Templates, gestufte Hilfen, eine kuratierte Aufgaben- und Wissensbasis sowie ein Evaluationsdatensatz entstehen.

Der wissenschaftliche Beitrag liegt insbesondere in der systematischen Untersuchung der Aufgabenteilung zwischen symbolischer Mathematik und generativer Sprachverarbeitung. Während STACK beziehungsweise Maxima die mathematische Prüfung übernehmen, soll das LLM didaktische Hinweise auf Grundlage kontrollierter Prüfergebnisse formulieren.

Der erwartete Beitrag umfasst:

1. ein prototypisches Integrationskonzept für digitale Mathematikaufgaben, symbolische Prüfung und LLM-Tutoring
2. eine Untersuchung der Anbindung von Maxima beziehungsweise Headless STACK an einen Tutor-Workflow
3. einen Vergleich zwischen PRT-basierter Fehlerdiagnose und optionaler dynamischer symbolischer Analyse
4. einen Prompt- und Workflow-Ansatz für gestufte Hilfen
5. eine Bewertung selbst gehosteter LLMs für den betrachteten Tutor-Anwendungsfall
6. eine technische und fachlich-didaktische Evaluation anhand typischer Fehlerantworten
7. Gestaltungsempfehlungen für zukünftige LLM-basierte Mathematiktutoren

◀ 7 Quellenlage / aktueller Stand der Technik

Die Arbeit liegt an der Schnittstelle von mathematischem E-Assessment, intelligenten Tutorsystemen, generativer KI, neuro-symbolischen Systemen und formativem Feedback. Die Quellenlage umfasst daher sowohl etablierte Forschung zu automatisierter mathematischer Bewertung und Tutorhinweisen als auch aktuelle Arbeiten zu LLM-basierten Mathematiktutoren, Retrieval-Augmented Generation und symbolischer Verifikation.

7.1 Mathematisches E-Assessment und STACK

STACK ermöglicht die symbolische Bewertung mathematischer Antworten mit Maxima [San13; STA26]. Potential Response Trees erlauben es, typische Fehlermuster durch von Lehrenden definierte symbolische Tests zu identifizieren und differenziertes Feedback auszugeben. Die Forschung zu STACK hebt die mathematische Ausdrucksstärke und die Möglichkeiten formativer Rückmeldung hervor, verweist jedoch zugleich auf den Aufwand für Aufgabenerstellung, PRT-Modellierung und Pflege der Feedbacklogik.

Für die geplante Arbeit ist STACK sowohl als Ausgangssystem für digitale Aufgaben als auch als mögliche Headless-Prüfkomponente relevant. Zu untersuchen ist, wie vorhandene PRT-Diagnosen für einen LLM-Tutor nutzbar gemacht werden können und ob eine zusätzliche dynamische Anbindung von Maxima oder Headless STACK einen Mehrwert bietet.

7.2 LLM-basierte Mathematiktutoren und Hint-Generierung

Aktuelle Arbeiten untersuchen LLMs zur mathematischen Fragebeantwortung, zur Erzeugung von Feedback und zur Generierung gestufter Hinweise [Ale+23; Lev+23]. Eine zentrale Herausforderung besteht darin, Lernende zu unterstützen, ohne die vollständige Lösung vorwegzunehmen. Aus der Forschung zu intelligenten Tutorsystemen sind Hint-Sequenzen bekannt, bei denen Hinweise schrittweise konkreter werden [Van06; Sta+10]. Diese Ansätze bilden eine Grundlage für den in dieser Arbeit geplanten Prompt-Workflow.

◀7 Quellenlage / aktueller Stand der Technik

Forschung zu LLM-generierten Mathematikhinweisen zeigt zugleich, dass solche Hinweise nicht durchgängig zuverlässig genug sind, um ungeprüft an Lernende ausgegeben zu werden. Lehrendenkontrolle, symbolische Verifikation, strukturierte Prompt-Regeln und aufgabenspezifischer Kontext werden daher als wichtige Absicherungen betrachtet [Kas+23; NMC23].

7.3 Retrieval-Augmented Generation und Kontextsteuerung

Retrieval-Augmented Generation verbindet die Generierung eines Sprachmodells mit extern abgerufenen Wissensbeständen [Lew+20]. Arbeiten zur mathematischen Fragebeantwortung zeigen, dass RAG die fachliche und curriculare Verankerung verbessern kann [Lev+23]. Zugleich besteht ein Zielkonflikt zwischen starker Bindung an abgerufene Materialien und der sprachlichen beziehungsweise didaktischen Qualität der Antwort. Eine hohe Groundedness ist nicht automatisch mit hoher Korrektheit, Relevanz oder Hilfreichkeit gleichzusetzen.

Anwendungsorientierte Erweiterungen wie RAG+ ergänzen das abgerufene Fachwissen um Beispiele seiner konkreten Anwendung und zeigen Verbesserungen bei mathematischen Reasoning-Aufgaben [Wan+25]. Für den begrenzten Prototyp ist jedoch keine umfangreiche Vektordatenbank vorgesehen. Stattdessen sollen Aufgaben-ID, Diagnosecode und Hilfestufe zur deterministischen Auswahl kuratierter Kontextinformationen verwendet werden.

7.4 Symbolische Verifikation und neuro-symbolische Ansätze

Aktuelle Arbeiten kombinieren LLMs und RAG mit Computer-Algebra-Systemen, Code-Ausführungsumgebungen oder symbolischen Solvern [IDS23; UGC23; Wan26]. Dabei werden generierte Rechenschritte oder Endergebnisse beispielsweise durch SymPy geprüft [Meu+17]. Solche Arbeiten zeigen, dass Retrieval allein symbolische Ausführungsfehler nicht zuverlässig verhindert und eine deterministische mathematische Kontrollkomponente sinnvoll ist.

Die geplante Arbeit knüpft an diese neuro-symbolischen Ansätze an, setzt jedoch einen anderen Schwerpunkt. Das symbolische System soll nicht nur nachträglich eine vom LLM erzeugte Lösung prüfen. Vielmehr sollen STACK, PRTs, Maxima oder Headless STACK die Studierendenantwort bewerten beziehungsweise diagnostizieren und dem LLM eine kontrollierte Grundlage für die didaktische Hinweisgenerierung liefern.

◀7 Quellenlage / aktueller Stand der Technik

7.5 Kommerzielle und institutionelle Systeme

Es existieren verschiedene Systeme zur mathematischen Unterstützung:

- allgemeine LLM-basierte Chatbots wie *ChatGPT*, *Claude*, *Gemini*, *Microsoft Copilot* und *Perplexity*
- mathematikspezifische Tutor- und Solver-Systeme wie *Photomath*, *Symbolab*, *Mathway*, *WolframAlpha Pro* und *Microsoft Math Solver*
- E-Assessment-Systeme wie *Möbius*, *ALEKS*, *MyMathLab*, *WirisQuizzes*, *STACK* und *WeBWorK*
- adaptive Tutoring-Systeme und Lernplattformen wie *Khanmigo*, *Carnegie Learning/MATHia*, *Squirrel AI* und *Querium*
- institutionelle Hochschulangebote wie *KI & ME* der Hochschule Magdeburg-Stendal

Diese Systeme decken Teilaspekte wie Lösungserzeugung, symbolische Berechnung, adaptive Lernpfade, automatische Bewertung oder dialogische Unterstützung ab. Eine transparente und selbst gehostete Kombination aus STACK-/Maxima-Prüfung, explizitem Tutor-Workflow und didaktisch kontrollierter LLM-Hinweisgenerierung ist jedoch nicht als allgemeine Standardlösung etabliert.

7.6 Forschungslücke

Die allgemeine Kombination von LLMs, RAG und symbolischen Solvern ist bereits Gegenstand aktueller Forschung. Die Forschungslücke kann daher nicht in der erstmaligen Verbindung dieser Technologien gesehen werden.

Weniger untersucht ist hingegen die gezielte Nutzung einer symbolischen Bewertung oder Fehlerdiagnose von Studierendenantworten zur Steuerung eines didaktisch ausgerichteten LLM-Tutors. Insbesondere die Verbindung von:

- digitalen Mathematikaufgaben
- PRT-basierten Diagnosecodes
- Maxima oder Headless STACK als mathematischer Prüfinstanz
- einem selbst gehosteten LLM
- einem expliziten Tutor-Workflow mit Hilfestufen
- und der systematischen Vermeidung von Lösungsverrat

ist bislang nur unzureichend untersucht. Genau diese Kopplung bildet den zentralen Untersuchungsgegenstand der Arbeit.

8 Zeitplan

Ein möglicher Zeitplan für eine Bearbeitungsdauer von 19 Wochen ist in Tabelle 8.1 dargestellt.

Die schriftliche Dokumentation erfolgt parallel zur Entwicklung und Evaluation. Falls einzelne optionale Bestandteile, insbesondere die Agentenanbindung oder die Think-Aloud-Erprobung, nicht innerhalb des Zeitrahmens realisierbar sind, wird der Schwerpunkt auf den deterministischen Workflow und die technische sowie fachlich-didaktische Evaluation gelegt.

◀8 Zeitplan

Tabelle 8.1: Vorläufiger Zeitplan

Phase		Zeitraum	Inhalt
Literatur- und Marktanalyse		Woche 1–3	Recherche zu STACK, Maxima, Headless STACK, LLM-Tutoring, symbolischer Verifikation, Prompt-Design, kommerziellen und institutionellen Systemen
Anforderungsanalyse und Eingrenzung		Woche 4	Präzisierung der Architektur, Evaluationsfragen, Aufgabentypen und Mindestanforderungen
Aufgaben- und Fehlerkorpus		Woche 5–6	Auswahl von Aufgaben aus der Differentialrechnung; Definition typischer Fehler; Prüfung der Verfügbarkeit anonymisierter realer Studierendenantworten
Technische Basisintegration		Woche 7–8	Informationsübergabe aus STACK; Aufbau des Tutor-Interfaces; Validierung der Parameter und Kontextverwaltung
Maxima-/STACK-Anbindung		Woche 9–10	Implementierung und Test einer Maxima-beziehungsweise Headless-STACK-Schnittstelle; optionaler Vergleich mit PRT-Diagnosen
n8n- und LLM-Workflow		Woche 11–12	Orchestrierung des Workflows; Anbindung des selbst gehosteten LLMs; Fehlerbehandlung und Protokollierung
Prompt- und Tutor-Design		Woche 13–14	Entwicklung und Erprobung des gestuften Prompt-Workflows; Regeln gegen Lösungsverrat; Auswahl des Modells
Technische Evaluation		Woche 15–16	Tests mit Aufgaben und typischen Fehlerantworten; Messung von Prüfergebnissen, Stabilität und Antwortzeiten
Didaktische Evaluation		Woche 17	Expertenbewertung und, sofern realisierbar, explorative Think-Aloud-Erprobung
Auswertung und Diskussion		Woche 18	Auswertung der Ergebnisse; Beantwortung der Forschungsfragen; Ableitung von Gestaltungsempfehlungen
Abschluss		Woche 19	Fertigstellung, Überarbeitung, formale Prüfung und Abgabe der Arbeit

◀ 9 Vorläufige Gliederung

1. Einleitung

- a) Motivation und Problemstellung
- b) Zielsetzung
- c) Forschungsfragen
- d) Abgrenzung
- e) Aufbau der Arbeit

2. Theoretische und technische Grundlagen

- a) Digitale Mathematikaufgaben und E-Assessment
- b) Moodle, STACK und Potential Response Trees
- c) Maxima und Headless STACK
- d) Large Language Models und selbst gehostete Modellinferenz
- e) LLM-basierte Mathematikutoren und Hint-Generierung
- f) Retrieval-Augmented Generation und Kontextanreicherung
- g) Neuro-symbolische Systeme und symbolische Verifikation
- h) Formatives Feedback und gestufte Hilfen

3. Stand der Forschung und Technik

- a) Wissenschaftliche Arbeiten zu LLM-Tutoring
- b) RAG für mathematische Lernunterstützung
- c) Symbolische Solver in LLM-Systemen
- d) Kommerzielle und institutionelle Angebote
- e) Forschungslücke

4. Anforderungsanalyse und Konzeption

- a) Fachliche Eingrenzung auf Differentialrechnung
- b) Funktionale und didaktische Anforderungen

◀9 Vorläufige Gliederung

- c) Systemarchitektur und Datenfluss
- d) Aufgaben- und Kontextmodell
- e) PRT-basierte und dynamische Diagnose
- f) Tutor-Workflow und Hilfestufen
- g) Sicherheits- und Datenschutzaspekte

5. Prototypische Implementierung

- a) STACK-Anbindung und Parameterübergabe
- b) Tutor-Interface
- c) Anbindung von Maxima beziehungsweise Headless STACK
- d) n8n-Workflow
- e) Anbindung des selbst gehosteten LLMs
- f) Prompt- und Tutor-Policy
- g) Fehlerbehandlung und Protokollierung

6. Evaluation

- a) Evaluationsdesign und Aufgabensatz
- b) Typische und reale Studierendenfehler
- c) Technische Evaluation
- d) Vergleich der Diagnoseansätze
- e) Fachlich-didaktische Bewertung
- f) Optionale Think-Aloud-Erprobung

7. Ergebnisse und Diskussion

- a) Ergebnisse der technischen Umsetzung
- b) Qualität und Zuverlässigkeit der mathematischen Prüfung
- c) Qualität der generierten Tutorhinweise
- d) Einhaltung der Hilfestufen und Lösungserrat
- e) Eignung des selbst gehosteten LLMs
- f) Grenzen des Prototyps

8. Fazit und Ausblick

- a) Beantwortung der Forschungsfragen
- b) Zentrale Erkenntnisse
- c) Gestaltungsempfehlungen
- d) Ausblick auf produktive Moodle-Integration, größere Aufgabenbestände und weiterführende Agentenarchitekturen

Literaturverzeichnis

- [Ale+23] V. Aleven, R. Baraniuk, E. Brunskill, S. Crossley, D. Demszky, S. Fancsali, S. Gupta, K. Koedinger, C. Piech, S. Ritter u. a. „Towards the Future of AI-Augmented Human Tutoring in Math Learning“. In: *Artificial Intelligence in Education: Posters and Late Breaking Results, Workshops and Tutorials, Industry and Innovation Tracks, Practitioners, Doctoral Consortium and Blue Sky*. Cham: Springer Nature Switzerland, 2023, S. 26–31. DOI: [10.1007/978-3-031-36336-8_3](https://doi.org/10.1007/978-3-031-36336-8_3).
- [ES93] K. A. Ericsson und H. A. Simon. *Protocol Analysis: Verbal Reports as Data*. Revised edition. Cambridge, MA: MIT Press, 1993. ISBN: 9780262550239.
- [Gao+23] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun und H. Wang. „Retrieval-Augmented Generation for Large Language Models: A Survey“. In: *arXiv preprint arXiv:2312.10997* (2023). DOI: [10.48550/arXiv.2312.10997](https://doi.org/10.48550/arXiv.2312.10997). arXiv: [2312.10997](https://arxiv.org/abs/2312.10997) [cs.CL].
- [HT07] J. Hattie und H. Timperley. „The Power of Feedback“. In: *Review of Educational Research* 77.1 (2007), S. 81–112. DOI: [10.3102/003465430298487](https://doi.org/10.3102/003465430298487).
- [Hoc26] Hochschule Magdeburg-Stendal. *KI & ME*. 2026. URL: <https://kiandme.h2.de/> (besucht am 24.07.2026).
- [IDS23] S. Imani, L. Du und H. Shrivastava. „MathPrompter: Mathematical Reasoning Using Large Language Models“. In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics, Volume 3: Industry Track*. 2023, S. 37–42. DOI: [10.18653/v1/2023.acl-industry.4](https://doi.org/10.18653/v1/2023.acl-industry.4). URL: <https://aclanthology.org/2023.acl-industry.4/>.
- [Ji+23] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto und P. Fung. „Survey of Hallucination in Natural Language Generation“. In: *ACM Computing Surveys* 55.12 (2023), S. 1–38. DOI: [10.1145/3571730](https://doi.org/10.1145/3571730).

<9 Vorläufige Gliederung

- [Kas+23] E. Kasneci, K. Sessler, S. Küchemann, M. Bannert, D. Dementieva, F. Fischer, U. Gasser, G. Groh, S. Günnemann, E. Hüllermeier u. a. „ChatGPT for Good? On Opportunities and Challenges of Large Language Models for Education“. In: *Learning and Individual Differences* 103 (2023), S. 102274. DOI: [10.1016/j.lindif.2023.102274](https://doi.org/10.1016/j.lindif.2023.102274).
- [Kha26] Khan Academy. *Khanmigo*. 2026. URL: <https://www.khanmigo.ai/> (besucht am 24.07.2026).
- [Koc+22] E. Kochmar, D. D. Vu, R. Belfer, V. Gupta, I. V. Serban und J. Pineau. „Automated Data-Driven Generation of Personalized Pedagogical Interventions in Intelligent Tutoring Systems“. In: *International Journal of Artificial Intelligence in Education* 32.2 (2022), S. 323–349. DOI: [10.1007/s40593-021-00267-x](https://doi.org/10.1007/s40593-021-00267-x).
- [Lev+23] Z. Levonian, C. Li, W. Zhu, A. Gade, O. Henkel, M.-E. Postle und W. Xing. „Retrieval-Augmented Generation to Improve Math Question-Answering: Trade-offs Between Groundedness and Human Preference“. In: *NeurIPS 2023 Workshop on Generative AI for Education*. New Orleans, USA, 2023. DOI: [10.48550/arXiv.2310.03184](https://arxiv.org/abs/2310.03184). arXiv: [2310.03184](https://arxiv.org/abs/2310.03184) [cs.CL]. URL: <https://arxiv.org/abs/2310.03184>.
- [Lew+20] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel und D. Kiela. „Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks“. In: *Advances in Neural Information Processing Systems*. Bd. 33. 2020, S. 9459–9474. URL: <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html> (besucht am 24.07.2026).
- [Man+20] M. Maniktala, C. Cody, T. Barnes und M. Chi. „Avoiding Help Avoidance: Using Interface Design Changes to Promote Unsolicited Hint Usage in an Intelligent Tutor“. In: *International Journal of Artificial Intelligence in Education* 30.4 (2020), S. 637–667. DOI: [10.1007/s40593-020-00213-3](https://doi.org/10.1007/s40593-020-00213-3).
- [Map26] Maplesoft. *Möbius*. 2026. URL: <https://www.digitaled.com/mobius> (besucht am 24.07.2026).
- [Meu+17] A. Meurer, C. P. Smith, M. Paprocki, O. Bertik, S. B. Kirpichev, M. Rocklin, A. Kumar, S. Ivanov, J. K. Moore, S. Singh u. a. „SymPy: Symbolic Computing in Python“. In: *PeerJ Computer Science* 3 (2017), e103. DOI: [10.7717/peerj-cs.103](https://doi.org/10.7717/peerj-cs.103).

<9 Vorläufige Gliederung

- [Mia+23] G. Mialon, R. Dessì, M. Lomeli, C. Nalmpantis, R. Pasunuru, R. Raileanu, B. Rozière, T. Schick, J. Dwivedi-Yu, A. Celikyilmaz u. a. „Augmented Language Models: A Survey“. In: *arXiv preprint arXiv:2302.07842* (2023). DOI: [10.48550/arXiv.2302.07842](https://doi.org/10.48550/arXiv.2302.07842). arXiv: [2302.07842](https://arxiv.org/abs/2302.07842) [cs.CL].
- [n8n26] n8n GmbH. *n8n Documentation*. 2026. URL: <https://docs.n8n.io/> (besucht am 24.07.2026).
- [NM06] D. J. Nicol und D. Macfarlane-Dick. „Formative Assessment and Self-Regulated Learning: A Model and Seven Principles of Good Feedback Practice“. In: *Studies in Higher Education* 31.2 (2006), S. 199–218. DOI: [10.1080/03075070600572090](https://doi.org/10.1080/03075070600572090).
- [NMC23] B. D. Nye, D. Mee und M. G. Core. „Generative Large Language Models for Dialog-Based Tutoring: An Early Consideration of Opportunities and Concerns“. In: *Proceedings of the AIED 2023 Workshop on Empowering Education with LLMs*. 2023. URL: <https://ceur-ws.org/Vol-3487/paper4.pdf> (besucht am 24.07.2026).
- [RH10] L. Razzaq und N. T. Heffernan. „Hints: Is It Better to Give or Wait to Be Asked?“. In: *Intelligent Tutoring Systems*. Berlin und Heidelberg: Springer, 2010, S. 349–358. DOI: [10.1007/978-3-642-13388-6_39](https://doi.org/10.1007/978-3-642-13388-6_39).
- [San13] C. J. Sangwin. *Computer Aided Assessment of Mathematics*. Oxford: Oxford University Press, 2013. ISBN: 9780199660353. DOI: [10.1093/acprof:oso/9780199660353.001.0001](https://doi.org/10.1093/acprof:oso/9780199660353.001.0001).
- [Shu08] V. J. Shute. „Focus on Formative Feedback“. In: *Review of Educational Research* 78.1 (2008), S. 153–189. DOI: [10.3102/0034654307313795](https://doi.org/10.3102/0034654307313795).
- [SBS94] M. W. van Someren, Y. F. Barnard und J. A. C. Sandberg. *The Think Aloud Method: A Practical Guide to Modelling Cognitive Processes*. London: Academic Press, 1994. ISBN: 9780127142707.
- [STA26] STACK Project. *STACK Documentation*. Documentation of the STACK computer-aided assessment system. 2026. URL: <https://docs.stack-assessment.org/> (besucht am 24.07.2026).
- [Sta+10] J. C. Stamper, T. Barnes, L. Lehmann und M. Croy. „Enhancing the Automatic Generation of Hints with Expert Seeding“. In: *Intelligent Tutoring Systems*. Berlin und Heidelberg: Springer, 2010.
- [UGC23] S. Upadhyay, E. Ginsberg und C. Callison-Burch. „Improving Mathematics Tutoring with a Code Scratchpad“. In: *Proceedings of the 18th Workshop on Innovative Use of NLP for Building Educational Applications*. 2023, S. 20–28. DOI: [10.18653/v1/2023.bea-1.2](https://doi.org/10.18653/v1/2023.bea-1.2).

<9 Vorläufige Gliederung

- [Van06] K. VanLehn. „The Behavior of Tutoring Systems“. In: *International Journal of Artificial Intelligence in Education* 16.3 (2006), S. 227–265.
- [Wan26] J. Wang. „Low-Hallucination RAG for Mathematical Question Answering: A Trustworthy Generation Method Based on Retrieval Hygiene, Step-Tree Reasoning, and Symbolic-Solver Feedback“. In: *Proceedings of the 2nd International Conference on Generative Artificial Intelligence and Digital Media*. GADM 2026. New York, NY, USA: Association for Computing Machinery, 2026, S. 160–164. DOI: [10.1145/3803730.3803757](https://doi.org/10.1145/3803730.3803757). URL: <https://doi.org/10.1145/3803730.3803757>.
- [Wan+25] Y. Wang, S. Zhao, Z. Wang, M. Fan, X. Zhang, Y. Zhang, Z. Wang, H. Huang und T. Liu. „RAG+: Enhancing Retrieval-Augmented Generation with Application-Aware Reasoning“. In: *arXiv pre-print arXiv:2506.11555* (2025). Version 4, 23 September 2025. arXiv: [2506.11555](https://arxiv.org/abs/2506.11555) [cs.AI]. URL: <https://arxiv.org/abs/2506.11555> (besucht am 24.07.2026).
- [Wol26] Wolfram Research. *WolframAlpha*. 2026. URL: <https://www.wolframalpha.com/> (besucht am 24.07.2026).